

Assingment —2

Task 1: Statistical Summary for Survey Data

Prompt: generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values. **Code:**

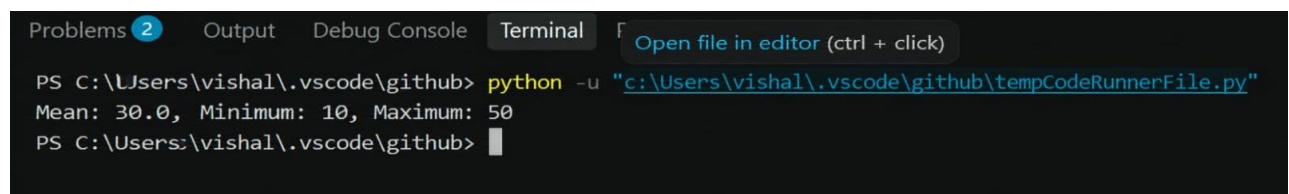
```
def calculate_statistics(numbers):    if not
numbers:

    return None, None, None # Return None for mean, min, and max if the list is empty

    mean_value = sum(numbers) / len(numbers)
min_value = min(numbers)    max_value = max(numbers)

    return mean_value, min_value, max_value #exapmple usage:
numbers = [10, 20, 30, 40, 50] mean, minimum, maximum = calculate_statistics(numbers)
print(f"Mean: {mean}, Minimum:
{minimum}, Maximum: {maximum}")
```

outpu:

A screenshot of a VS Code terminal window. The terminal has tabs for 'Problems' (with a blue circle containing the number 2), 'Output', 'Debug Console', and 'Terminal'. The 'Terminal' tab is active, showing a command prompt window with the following text: 'PS C:\Users\vishal\.vscode\github> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"', 'Mean: 30.0, Minimum: 10, Maximum: 50', and 'PS C:\Users\vishal\.vscode\github>'. A button labeled 'Open file in editor (ctrl + click)' is visible in the top right corner of the terminal window.

```
Problems 2 Output Debug Console Terminal Open file in editor (ctrl + click)
PS C:\Users\vishal\.vscode\github> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
Mean: 30.0, Minimum: 10, Maximum: 50
PS C:\Users\vishal\.vscode\github>
```

Task 2: Armstrong Number – AI Comparison

Prompt: Generate an Armstrong number checker using Given a number, check if it is an Armstrong number or not. Compare their outputs, logic style, and clarity.

Code:

```
def is_armstrong_number(n):  
    # Convert the number to a string to easily iterate over each digit    digits  
    = str(n)    num_digits = len(digits)  
  
    # Calculate the sum of each digit raised to the power of the number of digits  
    armstrong_sum = sum(int(digit) ** num_digits for digit in digits)  
  
    # Check if the calculated sum is equal to the original number    return  
    armstrong_sum == n # Example usage: number = 153    if  
is_armstrong_number(number):  
    print(f"{number} is an Armstrong number.") else:  
    print(f"{number} is not an Armstrong number.") output:
```

```
PS C:\Users\vishal\.vscode\github> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"  
153 is an Armstrong number.  
PS C:\Users\vishal\.vscode\github> █
```

Task 3: Leap Year Validation Using Cursor AI

Prompt: generate a Python program that checks whether a given year is a leap year.also give two different methods to check for leap year and compare their outputs, logic style, and clarity.

Code:

```
# Method 1: Using the standard leap year rules def is_leap_year_method1(year):  
if (year % 4 == 0 and year %  
100 != 0) or (year % 400 == 0):
```

```

        return True    else:

        return False

# Example usage:
year = 2020 if is_leap_year_method1(year):

print(f"{year} is a leap year.") else:    print(f"{year}
is not a leap year.")

# Method 2: Using a more concise approach def is_leap_year_method2(year):

    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
# Example usage: year = 2021    if is_leap_year_method2(year):

    print(f"{year} is a leap year.")    else:

print(f"{year} is not a leap year.")

#example usage:

year = 2024 if is_leap_year_method1(year) == is_leap_year_method2(year):

    print(f"Both methods agree that {year} is a leap year: {is_leap_year_method1(year)}")

```

output

:

```

PS C:\Users\vishal\.vscode\github> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
2020 is a leap year.
2021 is not a leap year.
Both methods agree that 2024 is a leap year: True
PS C:\Users\vishal\.vscode\github> █

```

Task 4: Student Logic (Odd/Even Sum)

Prompt: Write a Python program that calculates the sum of odd and even numbers in a tuple.

Code:

```
def sum_odd_even(numbers):  
    odd_sum = sum(x for x in numbers if x % 2 != 0)    even_sum  
= sum(x for x in numbers if x % 2 == 0)    return odd_sum,  
even_sum
```

Example usage:

```
tuple_numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10) odd_total,  
even_total = sum_odd_even(tuple_numbers) print(f"Sum of  
odd numbers: {odd_total}") print(f"Sum of even numbers:  
{even_total}")
```

output

:

```
PS C:\Users\vishal\.vscode\github> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"  
Sum of odd numbers: 25  
Sum of even numbers: 30  
PS C:\Users\vishal\.vscode\github> █
```